1 Der Standard Kalman-Filter für 2D-Feature-Tracking

Der Standard Kalman-Filter (KF) ist ein optimaler rekursiver Schätzer für lineare Systeme unter Einfluss von Gaußschem Rauschen. Für das Tracking von Keypoints im Bildraum (u, v) mittels eines Constant-Velocity (CV) Modells definieren wir den Zustandsvektor $\mathbf{x}_k$ zum Zeitpunkt k als:

$$\mathbf{x}_k = \begin{bmatrix} u_k \\ v_k \\ \dot{u}_k \\ \dot{v}_k \end{bmatrix} \tag{1}$$

Der Filter arbeitet nun in zwei rekursiven Schritten: Prädiktion (*a priori* Schätzung) und Korrektur (*a posteriori* Schätzung). Wir verwenden die Notation $\mathbf{x}_{k|k-1}$, was bedeutet: „Der geschätzte Zustand zum Zeitpunkt k, basierend auf dem Wissen bis zum Zeitpunkt $k-1$“.

1.1 Prädiktion (Predict)

In diesem Schritt wird der Zustand basierend auf dem Systemmodell in die Zukunft projiziert. Die Matrix F beschreibt die lineare Kinematik (bei konstanter Geschwindigkeit gilt $u_k = u_{k-1} + \dot{u}_{k-1}\Delta t$). Q ist die Prozessrauschkovarianz (Modellunsicherheit).

1. Zustand $\mathbf{x}_{k|k-1}$ prädizieren:

$$\mathbf{x}_{k|k-1} = F\mathbf{x}_{k-1|k-1} \tag{2}$$

2. Fehlerkovarianz $P_{k|k-1}$ prädizieren (Unsicherheit unserer Schätzung):

$$P_{k|k-1} = FP_{k-1|k-1}F^T + Q \tag{3}$$

Für das 2D-CV-Modell mit der Zeitschrittweite Δt gilt die Systemmatrix F:

$$F = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4}$$

Q (Prozessrauschkovarianz): Dies ist ein Tuning-Parameter. Er modelliert externe Einflüsse, die unser CV-Modell verletzen (z. B. plötzliche Beschleunigungen des Objekts oder der Kamera). Je höher Q, desto weniger vertraut der Filter seinem eigenen physikalischen Modell.

1.2 Korrektur (Update)

Sobald eine neue Messung (nur die Pixelkoordinaten, nicht die Geschwindigkeiten)

$$\mathbf{z}_k = \begin{bmatrix} u_{\text{gemessen}} \\ v_{\text{gemessen}} \end{bmatrix} \tag{5}$$

(z. B. von *xfeat*) vorliegt, wird die Prädiktion korrigiert. Die Messmatrix H projiziert den Zustandsraum in den Messraum, R ist die Messrauschkovarianz (Unsicherheit der Kamera/des Matchers).

1. Innovation (Residuum) berechnen:

$$\tilde{\mathbf{y}}_k = \mathbf{z}_k - H\mathbf{x}_{k|k-1} \tag{6}$$

   - H (Messmatrix): Übersetzt den theoretischen Zustand in den Messraum (blendet hier die Geschwindigkeiten aus, da die Kamera sie nicht direkt misst).

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \tag{7}$$

   - $\tilde{\mathbf{y}}_k$: Ist die Differenz zwischen dem, was *xfeat* sieht, und dem, was der Filter erwartet hat.

2. Innovationskovarianz berechnen:

$$S_k = HP_{k|k-1}H^T + R \tag{8}$$

   - R (Messrauschkovarianz): Beschreibt, wie stark die Keypoints von xfeat zittern. Das ist dein zweiter großer Tuning-Parameter. Ist R groß, vertraut der Filter der Messung wenig.

3. Kalman-Verstärkung (Kalman Gain) berechnen:

$$K_k = P_{k|k-1}H^T S_k^{-1} \tag{9}$$

   - K_k: Das ist das "Herzstück". Der Kalman-Gain bestimmt, wie stark das Residuum $\tilde{\mathbf{y}}_k$ gewichtet wird. Ist die Messung sehr sicher (R klein), wird K groß. Ist das Modell sehr sicher (P klein), wird K klein.

4. Zustand aktualisieren:

$$\mathbf{x}_{k|k} = \mathbf{x}_{k|k-1} + K_k\tilde{\mathbf{y}}_k \tag{10}$$

   - $\mathbf{x}_{k|k}$: Das ist dein final geglätteter Wert, den du an deine IBVS-Regelung weitergibst.

5. Fehlerkovarianz aktualisieren:

$$P_{k|k} = (I - K_kH)P_{k|k-1} \tag{11}$$

   - Die Unsicherheit sinkt, da nun eine neue Messung verarbeitet wurde. (I ist die Einheitsmatrix).

1.3 Bedeutung das für das IBVS-Projekt und Limitationen

Wenn *xfeat* ausfällt (Okklusion), wird der Update-Schritt übersprungen und nutzt für ein paar Frames nur den Predict-Schritt ($\mathbf{x}_{k|k} = \mathbf{x}_{k|k-1}$). Der Roboter folgt dem Objekt mit der zuletzt berechneten konstanten Geschwindigkeit weiter, bis xfeat den Keypoint wiederfindet.

Das obige Modell funktioniert für ein simples Tracking im 2D-Bildraum. Im IBVS-Projekt muss jedoch die 3D-Welt, die Kamerabewegung oder die Roboterkinematik miteinbezogen werden. Dadurch stößt der konventionelle KF an ein Grenze: Dieser setzt strikt lineare System- (F) und Messmodelle (H) voraus.

In der realen Robotik und Computervision ist jedoch fast alles nichtlinear:

- Das Kameramodell (Pinhole Camera): Die Projektion eines 3D-Punktes (X,Y,Z) auf ein 2D-Pixel (u,v) erfordert eine Division durch die Tiefe Z. Eine Division durch eine Zustandsvariable ist eine nichtlineare Operation.
- Roboterkinematik: Wenn Gelenkwinkel in Endeffektor-Positionen (und damit Kamerapositionen) umgerechnet werden, wird Sinus und Kosinus genutzt (ebenfalls hochgradig nichtlinear).

Wenn eine Gauß-Verteilung (Kovarianz P) durch eine nichtlineare Funktion geleitet wird, ist das Ergebnis keine Gauß-Verteilung mehr. Der Standard-Filter würde schnell divergieren (die Spur verlieren).

1.4 Praktisches Tuning und Mahalanobis-Gating

Die Parametrisierung der Rauschmatrizen Q und R muss die physikalische Skalierung des Bildraums (Pixel) widerspiegeln.

- **Messrauschen R:** Wird empirisch auf Basis des Jitters des Feature-Matchers (z. B. *xfeat*) gesetzt. Bei Schwankungen von ca. ± 10 Pixeln ergibt sich eine Varianz von $R = 100.0$.
- **Prozessrauschen Q:** Da das 2D-CV-Modell die Roboterkinematik ignoriert, ist der Modellfehler bei Beschleunigungen enorm. Q muss entsprechend groß gewählt werden (z. B. $Q = 500.0$), damit der Filter der Messung ausreichend vertraut und Filter-Lag minimiert wird.
- **Mahalanobis-Gate:** Die 4 virtuellen Proxy-Punkte (2D) erzeugen einen Messvektor mit $m = 8$ Freiheitsgraden (DoF). Laut der χ^2-Verteilungstabelle liegt der Schwellenwert für 8 DoF bei einem 99 %-Konfidenzintervall bei exakt $\gamma = 20.09$. Dieser theoretische Wert erzielt in der Praxis äußerst robuste Ergebnisse bei der Ausreißerunterdrückung.

Der Schwellenwert γ wird aus der Umkehrfunktion der χ^2-Verteilung entnommen [1]. Die Anzahl der Freiheitsgrade (DoF) entspricht der Dimension des Messvektors z_k. In Tabelle 1 sind typische Gating-Schwellenwerte für das visuelle Tracking aufgeführt.

Freiheitsgrade (DoF)	γ **für 95 % Konfidenz**	γ **für 99 % Konfidenz**
2 (z. B. 1 Pixel)	5.99	9.21
4 (z. B. 2 Pixel)	9.49	13.28
6 (z. B. 3D Pose)	12.59	16.81
8 (z. B. 4 Proxy-Pixel)	15.51	**20.09**

Tabelle 1: Auszug aus der χ^2-Verteilungstabelle für das Measurement-Gating.

1.5 Tracking Recovery und Relokalisation

Ein robustes IBVS-System muss in der Lage sein, nach temporären Verdeckungen (Occlusions), Sensorfehlern oder manuellen Eingriffen (Teleportation des Roboters) die Objektverfolgung wieder aufzunehmen. Verbleibt der Kalman-Filter über eine längere Sequenz im Reject-Status (ausgelöst durch das Mahalanobis-Gate), divergiert die Prädiktion unweigerlich von der Realität.

Um ein *Deadlocking* des Filters zu verhindern, wird eine explizite Relokalisations-Logik (Tracking Recovery) implementiert [2]. Bei Reaktivierung des Regelkreises (oder nach einem Timeout von N verworfenen Frames) wird ein harter Reset des Filterzustands erzwungen:

1. Die Fehlerkovarianzmatrix wird auf einen initialen, sehr hohen Wert zurückgesetzt (z. B. $P = 1000 \cdot I$), was einer maximalen Unsicherheit des aktuellen Modells entspricht.

2. Der Zustandsvektor x wird direkt mit der aktuellen, ungefilterten Messung z_k (gewonnen aus der RANSAC-Homographie) überschrieben.

Durch diesen Eingriff verliert der Filter kurzzeitig seine Glättungseigenschaften, kann sich jedoch instantan an stark veränderte Umgebungsbedingungen anpassen und den Regelfehler für den IBVS-Controller wieder valide berechnen.

1.6 Systemverhalten bei Messausfällen (Drift-Phänomen)

Ein inhärentes Problem des Constant-Velocity-Modells zeigt sich bei langanhaltenden Ausreißern (z. B. durch starke Verdeckungen oder abrupte, unmodellierte Roboterbewegungen, die vom Gating abgelehnt werden).

Wird der Update-Schritt über mehrere Frames hinweg übersprungen, greift das System ausschließlich auf den Prädiktionsschritt zurück. Der Filter integriert die zuletzt geschätzte Pixel-Geschwindigkeit $(\dot{u}, \dot{v})$ unkorrigiert weiter. Dies führt zu einem linearen Drift der Proxy-Punkte im Bildraum. Während dieses Verhalten bei echten kurzzeitigen Verdeckungen (Occlusions) erwünscht ist, da die Objektbewegung prädiziert wird, führt es bei plötzlichen Stopps des Roboters zu einer starken Divergenz zwischen dem gefilterten Zustand und der realen Szene.

2 Der Extended Kalman Filter (EKF) und Linearisierung

Sobald das Systemmodell oder das Messmodell nichtlinear werden – was in der visuellen Robotik (IBVS) aufgrund von Kameramodellen und Rotationskinematik der Regelfall ist – versagt der Standard Kalman-Filter. Der Extended Kalman Filter (EKF) löst dies durch eine lokale Linearisierung mittels einer Taylorreihenentwicklung erster Ordnung.

Die konstanten Matrizen F und H werden durch nichtlineare Funktionen $f(x)$ und $h(x)$ ersetzt. Der EKF berechnet in jedem Zeitschritt k die zugehörigen Jacobi-Matrizen (partielle Ableitungen) F_k und H_k, evaluiert am aktuellen Schätzpunkt.

2.1 EKF Prädiktion und Update

Die Prädiktion des Zustands erfolgt direkt über die nichtlineare Systemfunktion f:

$$x_{k|k-1} = f(x_{k-1|k-1}, u_k) \tag{12}$$
$$P_{k|k-1} = F_k P_{k-1|k-1} F_k^T + Q \tag{13}$$

Die Korrektur nutzt die nichtlineare Messfunktion h für das Residuum und die Jacobi-Matrix H_k für die Kovarianz- und Gain-Berechnung:

$$\tilde{y}_k = z_k - h(x_{k|k-1}) \tag{14}$$
$$S_k = H_k P_{k|k-1} H_k^T + R \tag{15}$$
$$K_k = P_{k|k-1} H_k^T S_k^{-1} \tag{16}$$
$$x_{k|k} = x_{k|k-1} + K_k \tilde{y}_k \tag{17}$$
$$P_{k|k} = (I - K_k H_k) P_{k|k-1} \tag{18}$$

2.2 Beispiel: Jacobi-Matrix eines Lochkameramodells

Projiziert man einen 3D-Punkt $x = [X, Y, Z]^T$ auf eine Bildebene (u, v), lautet die nichtlineare Messfunktion $h(x)$ unter Annahme der Brennweiten f_x, f_y und des Hauptpunktes (c_x, c_y):

$$h(x) = \begin{bmatrix} f_x \frac{X}{Z} + c_x \\ f_y \frac{Y}{Z} + c_y \end{bmatrix} \tag{19}$$

Da die Division durch den Zustand Z eine Nichtlinearität darstellt, bilden wir die Jacobi-Matrix $H_k = \frac{\partial h}{\partial x}$:

$$H_k = \begin{bmatrix} \frac{\partial u}{\partial X} & \frac{\partial u}{\partial Y} & \frac{\partial u}{\partial Z} \\ \frac{\partial v}{\partial X} & \frac{\partial v}{\partial Y} & \frac{\partial v}{\partial Z} \end{bmatrix} = \begin{bmatrix} \frac{f_x}{Z} & 0 & -f_x \frac{X}{Z^2} \\ 0 & \frac{f_y}{Z} & -f_y \frac{Y}{Z^2} \end{bmatrix} \tag{20}$$

Diese Matrix H_k wird in jedem Frame mit den aktuellen Schätzwerten für X, Y und Z neu evaluiert und in den Update-Gleichungen verwendet.

3 Der Unscented Kalman Filter (UKF)

Der Extended Kalman Filter (EKF) verliert an Genauigkeit, wenn die System- oder Messmodelle stark nichtlinear sind, da die Linearisierung mittels Jacobi-Matrizen (Taylor-Entwicklung 1. Ordnung) nur lokal gültig ist. Der Unscented Kalman Filter (UKF), basierend auf der Arbeit von Julier und Uhlmann (1997), umgeht dieses Problem durch die *Unscented Transform* (UT).

Das Kernkonzept des UKF ist es, nicht die Funktion zu approximieren, sondern die Wahrscheinlichkeitsverteilung.

3.1 Die Unscented Transform (UT) und Sigma-Punkte

Für einen n-dimensionalen Zustandsvektor x mit Mittelwert μ und Kovarianz P wählt der UKF deterministisch $2n+1$ sogenannte Sigma-Punkte $\mathcal{X}^{(i)}$ aus. Diese Punkte repräsentieren die Verteilung exakt bis zur zweiten Ordnung (Mittelwert und Kovarianz).

Die Berechnung der Sigma-Punkte erfolgt typischerweise über die Matrixwurzel der Kovarianzmatrix (z. B. mittels Cholesky-Zerlegung):

$$\mathcal{X}^{(0)} = \mu \tag{21}$$

$$\mathcal{X}^{(i)} = \mu + \left(\sqrt{(n+\lambda)P}\right)_i \quad \text{für } i = 1, \ldots, n \tag{22}$$

$$\mathcal{X}^{(i)} = \mu - \left(\sqrt{(n+\lambda)P}\right)_{i-n} \quad \text{für } i = n+1, \ldots, 2n \tag{23}$$

Dabei ist λ ein Skalierungsparameter, der die Streuung der Sigma-Punkte um den Mittelwert steuert, und der Index i bezeichnet die i-te Spalte der Matrixwurzel.

Jedem Sigma-Punkt $\mathcal{X}^{(i)}$ werden Gewichte für den Mittelwert $W_m^{(i)}$ und für die Kovarianz $W_c^{(i)}$ zugeordnet.

3.2 Prädiktion und Update ohne Linearisierung

Anstatt Jacobi-Matrizen zu berechnen, werden die Sigma-Punkte direkt durch die nichtlinearen Funktionen f (Systemmodell) oder h (Messmodell) propagiert. Für das Messmodell (wie unser Kameramodell im IBVS) bedeutet das:

$$\mathcal{Z}^{(i)} = h(\mathcal{X}^{(i)}) \tag{24}$$

Der prädizierte Messmittelwert $\hat{z}$ und die zugehörige Kovarianz S ergeben sich dann einfach als gewichtete Summe der transformierten Punkte:

$$\hat{z} = \sum_{i=0}^{2n} W_m^{(i)} \mathcal{Z}^{(i)} \tag{25}$$

$$S = \sum_{i=0}^{2n} W_c^{(i)} (\mathcal{Z}^{(i)} - \hat{z})(\mathcal{Z}^{(i)} - \hat{z})^T + R \tag{26}$$

Fazit für IBVS-Anwendungen: Der UKF eliminiert die Notwendigkeit fehleranfälliger analytischer Ableitungen für komplexe Kameraprojektionen und Roboterkinematiken. Er liefert bei starken Nichtlinearitäten eine präzisere Schätzung als der EKF, ist jedoch rechnerisch oft etwas aufwendiger, da $2n+1$ Punkte pro Zeitschritt propagiert werden müssen.

4 Der Error-State Kalman Filter (ESKF)

Für Systeme, die 3D-Rotationen beinhalten (z. B. freifliegende Kameras im IBVS-Kontext), führt der Standard-EKF aufgrund der Überparametrisierung von Quaternionen (4 Parameter für 3 Freiheitsgrade) zu singulären Kovarianzmatrizen und der Verletzung der Einheitslängen-Bedingung bei der Zustandsaktualisierung. Der Error-State Kalman Filter (ESKF) löst dies, indem er den Zustand aufteilt:

$$x_t = x \oplus \delta x \tag{27}$$

Hierbei ist x_t der wahre Zustand, x der Nominalzustand (Integration der Systemkinematik ohne Rauschen) und δx der Fehlerzustand (Error-State). Der Operator $\oplus$ steht für die Zustandskomposition (Addition für Translationen, Multiplikation für Quaternionen). Der Filter schätzt *ausschließlich* den Fehlerzustand δx, der für Rotationen minimal parametrisiert werden kann (als 3D-Vektor $\delta\theta$).

4.1 ESKF Prädiktion (Predict)

Der Nominalzustand x wird analog zum EKF deterministisch und rauschfrei über das nichtlineare Systemmodell f propagiert:

$$x_{k|k-1} = f(x_{k-1|k-1}, u_k) \tag{28}$$

Die Prädiktion des Fehlerzustands ist im Erwartungswert Null ($\delta x_{k|k-1} = 0$). Die Fehler-Kovarianz P wächst jedoch basierend auf der Fehler-Kinematik:

$$P_{k|k-1} = F_{\delta x} P_{k-1|k-1} F_{\delta x}^T + F_i Q F_i^T \tag{29}$$

Dabei ist $F_{\delta x}$ die Jacobi-Matrix der zeitdiskreten Fehlerkinematik ($F_{\delta x} = \frac{\partial f_{\delta x}}{\partial \delta x}$) und F_i die Rausch-Eingangsmatrix, die das Prozessrauschen Q (z. B. IMU-Rauschen) korrekt in den Zustandsraum mappt.

4.2 ESKF Korrektur (Update) und die Kettenregel

Liegt eine Messung z_k vor, wird das Residuum $\tilde{y}_k$ bezüglich des unkorrigierten Nominalzustands mittels der Standard-Messfunktion $h(x)$ gebildet:

$$\tilde{y}_k = z_k - h(x) \tag{30}$$

Da der Kalman-Filter den Fehlerzustand schätzt, muss die Mess-Jacobi-Matrix $H_{\delta x}$ nach dem Fehlerzustand abgeleitet werden. Dies erfolgt über die Kettenregel:

$$H_{\delta x} = \frac{\partial h(x_t)}{\partial \delta x} = \underbrace{\frac{\partial h}{\partial x_t}}_{H_x} \cdot \underbrace{\frac{\partial x_t}{\partial \delta x}}_{X_{\delta x}} \tag{31}$$

- H_x: Die Standard-Mess-Jacobi-Matrix (wie im EKF, z. B. das Lochkameramodell).
- $X_{\delta x}$: Die Zusammensetzungs-Jacobi-Matrix (Jacobian of Composition). Sie bildet den minimalen Fehlerraum (z. B. 3D-Vektor) auf den vollen Zustandsraum (z. B. 4D-Quaternion) ab.

Mit $H_{\delta x}$ erfolgt das Standard-Update:

$$S_k = H_{\delta x} P_{k|k-1} H_{\delta x}^T + R \tag{32}$$

$$K_k = P_{k|k-1} H_{\delta x}^T S_k^{-1} \tag{33}$$

Der Filter schätzt nun den aktuellen Fehlerzustand und aktualisiert die Kovarianz:

$$\delta x_{k|k} = K_k \tilde{y}_k \tag{34}$$

$$P_{k|k} = (I - K_k H_{\delta x}) P_{k|k-1} \tag{35}$$

4.3 Injektion und Reset

Abschließend wird der geschätzte Fehler in den Nominalzustand injiziert, um diesen zu korrigieren. Da der Fehler nun Teil des Nominalzustands ist, wird der Fehlerzustand für den nächsten rekursiven Schritt zurückgesetzt (Reset):

$$x \leftarrow x \oplus \delta x_{k|k} \tag{36}$$

$$\delta x_{k|k} \leftarrow 0 \tag{37}$$

Dieser Reset-Mechanismus sorgt dafür, dass der Fehlerzustand stets nahe Null bleibt, wodurch die Linearisierungsfehler des EKF minimiert werden.

4.4 ESKF Beispiel: 3D-Orientierungstracking (Attitude Estimation)

Um den ESKF zu veranschaulichen, betrachten wir das reine Tracking der 3D-Kameraorientierung. Der Nominalzustand ist das Quaternion q, der Fehlerzustand der 3D-Rotationsvektor $\delta\theta$. Die Eingabe u ist die Winkelgeschwindigkeit ω eines Gyroskops. Die Messung z ist die Beobachtung eines bekannten 3D-Weltpunktes p_w im Kamerabild.

4.4.1 1. Prädiktion (Nominal- und Fehlerzustand)

Die physikalische Prädiktion des Nominalquaternions q erfolgt durch Integration der Winkelgeschwindigkeit ω_k über das Zeitintervall Δt:

$$q_{k|k-1} = q_{k-1|k-1} \otimes \exp\left(\frac{1}{2}\omega_k \Delta t\right) \tag{38}$$

Wobei $\otimes$ die Quaternionen-Multiplikation und exp() die Abbildung eines Rotationsvektors auf ein Quaternion darstellt.

Die Prädiktion des Fehlerzustands bleibt $\delta\theta_{k|k-1} = 0$. Die zugehörige 3×3 Kovarianzmatrix P wächst abhängig von der Fehlerkinematik $F_{\delta x}$ und dem Gyroskop-Rauschen Q:

$$P_{k|k-1} = \underbrace{(I_{3\times 3} - [\omega_k \times]\Delta t)}_{F_{\delta x}} P_{k-1|k-1} F_{\delta x}^T + Q\Delta t^2 \tag{39}$$

Hierbei ist $[\omega_k \times]$ die schiefsymmetrische Matrix des Vektors ω_k.

4.4.2 2. Korrektur (Update)

Zur Berechnung des Residuums projiziert das Nominalquaternion $q_{k|k-1}$ den Weltpunkt in das Kamera-Koordinatensystem, was den erwarteten Punkt $\hat{p}_c$ ergibt:

$$\tilde{y}_k = z_k - \hat{p}_c \tag{40}$$

Die ESKF-Mess-Jacobi-Matrix $H_{\delta x}$ wird nicht nach dem 4D-Quaternion abgeleitet, sondern direkt nach dem 3D-Fehlervektor $\delta\theta$. Die analytische Ableitung der Rotation eines Punktes nach dem Rotationsfehler entspricht dem negativen Kreuzprodukt des erwarteten Punktes:

$$H_{\delta x} = [\hat{p}_c \times] = \begin{bmatrix} 0 & -\hat{p}_{c,z} & \hat{p}_{c,y} \\ \hat{p}_{c,z} & 0 & -\hat{p}_{c,x} \\ -\hat{p}_{c,y} & \hat{p}_{c,x} & 0 \end{bmatrix} \tag{41}$$

Mit dieser kompakten 3×3 Matrix $H_{\delta x}$ wird der Standard-Update-Schritt ausgeführt, um den Fehlerzustand $\delta\theta_{k|k}$ zu schätzen:

$$S_k = H_{\delta x} P_{k|k-1} H_{\delta x}^T + R \tag{42}$$

$$K_k = P_{k|k-1} H_{\delta x}^T S_k^{-1} \tag{43}$$

$$\delta\theta_{k|k} = K_k \tilde{y}_k \tag{44}$$

$$P_{k|k} = (I_{3\times 3} - K_k H_{\delta x}) P_{k|k-1} \tag{45}$$

4.4.3 3. Injektion und Reset

Der berechnete 3D-Korrekturwinkel $\delta\theta_{k|k}$ wird als Fehler-Quaternion formuliert und auf den Nominalzustand aufmultipliziert (Injiziert):

$$q_{k|k} = q_{k|k-1} \otimes \exp\left(\frac{1}{2}\delta\theta_{k|k}\right) \tag{46}$$

Anschließend wird der Fehlerzustand für den nächsten Zyklus zurückgesetzt: $\delta\theta \leftarrow 0$.

5 Ausreißererkennung (Outlier Rejection) mittels Mahalanobis-Distanz

In IBVS-Anwendungen sind Feature-Matcher (wie *xfeat*) anfällig für falsche Assoziationen (False Positives) durch Hintergrundclutter, Reflexionen oder temporäre Verdeckungen. Die unbedarfte Integration solcher Ausreißer in den Update-Schritt des Kalman-Filters führt zur Divergenz der Zustandsschätzung und folglich zur Instabilität des Reglers.

Um dies zu verhindern, wird ein Gating-Verfahren angewendet, das Messungen vor dem Update-Schritt auf statistische Plausibilität prüft.

5.1 Die Mahalanobis-Distanz

Anstelle der unskalierten euklidischen Distanz im Bildraum wird die Mahalanobis-Distanz verwendet. Sie gewichtet das Residuum $\tilde{y}_k$ (die Differenz zwischen Messung und Prädiktion) mit der Innovationskovarianz S_k. Die Matrix S_k repräsentiert die kombinierte Unsicherheit aus Systemmodell und Messrauschen.

Die quadrierte Mahalanobis-Distanz D^2 berechnet sich als:

$$D^2 = \tilde{y}_k^T S_k^{-1} \tilde{y}_k \tag{47}$$

5.2 Gating mittels Chi-Quadrat-Test (χ^2)

Unter der Annahme von normalverteiltem Rauschen folgt die Variable D^2 einer χ^2-Verteilung mit m Freiheitsgraden (Degrees of Freedom, DoF), wobei m der Dimension der Messung entspricht. Für 2D-Feature-Tracking im Bildraum gilt $m = 2$.

Um eine Messung zu validieren, wird ein Konfidenzintervall (typischerweise 95 %) gewählt. Der entsprechende Schwellenwert γ wird aus der χ^2-Verteilungstabelle entnommen (für $m = 2, p = 0.95$ ist $\gamma \approx 5.991$).

Die Gating-Bedingung lautet:

$$\begin{cases} D^2 \leq \gamma & \text{Messung validiert (Update durchführen)} \\ D^2 > \gamma & \text{Ausreißer (Update überspringen)} \end{cases} \tag{48}$$

Wird eine Messung als Ausreißer klassifiziert, wird der Update-Schritt vollständig übersprungen. Der Filter geht in einen reinen Prädiktionsmodus über, bei dem die a-priori-Schätzungen als a-posteriori-Werte für den aktuellen Zeitschritt übernommen werden:

$$x_{k|k} = x_{k|k-1} \tag{49}$$

$$P_{k|k} = P_{k|k-1} \tag{50}$$

Dies ermöglicht es dem System, kurzzeitige Fehler oder Verdeckungen (Occlusions) des Keypoints robust zu überbrücken.

6 Praktisches Filter-Tuning: Bestimmung von Q und R

Die Leistungsfähigkeit des Kalman-Filters hängt maßgeblich von der korrekten Parametrisierung der Prozessrauschkovarianz Q und der Messrauschkovarianz R ab. Diese Matrizen steuern die Gewichtung (Kalman-Gain) zwischen dem physikalischen Prädiktionsmodell und den tatsächlichen Sensormessungen.

6.1 Messrauschen R (Sensormodell)

Die Matrix R beschreibt die Unsicherheit der Beobachtung. In einem IBVS-System mit Keypoint-Tracking repräsentiert sie den Pixel-Jitter des Matchers (z. B. *xfeat*). Sie lässt sich empirisch ermitteln, indem die Varianz σ^2 der Pixelkoordinaten (u, v) eines statischen Ziels über mehrere Frames gemessen wird. Unter der Annahme von unkorreliertem Rauschen in u- und v-Richtung ergibt sich eine Diagonalmatrix:

$$R = \begin{bmatrix} \sigma_u^2 & 0 \\ 0 & \sigma_v^2 \end{bmatrix} \tag{51}$$

Ein großes R führt zu starker Glättung (der Filter vertraut der Messung wenig), erhöht jedoch die Latenz (Lag).

6.2 Prozessrauschen Q (Systemmodell)

Die Matrix Q modelliert die Unsicherheit des Systemmodells, also Effekte, die analytisch nicht erfasst wurden. Bei Verwendung eines Constant-Velocity (CV) Modells repräsentiert Q die unmodellierten Beschleunigungen des Objekts. Die Bestimmung von Q ist in der Praxis oft heuristisch. Ein zu klein gewähltes Q führt zu einem trägen Filter, der echten Richtungswechseln des Objekts nicht schnell genug folgt (Filter-Lag oder Divergenz). Ein zu großes Q degradiert den Kalman-Filter zu einem reinen Beobachter, der das Messrauschen ungedämpft an den Regler weitergibt. Das Tuning ist ein Kompromiss (Trade-off) zwischen Glättung (Smoothness) und Reaktionsgeschwindigkeit (Responsiveness).

7 Der 1€ Filter (One Euro Filter)

Für das reine Glätten von 2D-Pixelkoordinaten in IBVS-Anwendungen, bei denen keine komplexe Sensorfusion oder physikalische Systemmodellierung erforderlich ist, stellt der 1€ Filter eine hocheffiziente Alternative zum Kalman-Filter dar. Er adressiert direkt den Trade-off zwischen Jitter (Zittern im Stillstand) und Lag (Latenz bei Bewegung).

7.1 Prinzip der adaptiven exponentiellen Glättung

Der 1€ Filter ist ein Tiefpassfilter erster Ordnung (exponentielle Glättung), dessen Glättungsfaktor α nicht konstant ist, sondern dynamisch an die Änderungsrate (Geschwindigkeit) des Signals angepasst wird. Die Grundgleichung lautet:

$$\hat{x}_i = \alpha x_i + (1 - \alpha)\hat{x}_{i-1} \tag{52}$$

Dabei ist x_i die verrauschte Eingabe zum Zeitschritt i und $\hat{x}_i$ der gefilterte Wert. Der Faktor $\alpha \in (0, 1]$ ergibt sich aus der Sampling-Periode T_e und einer dynamischen Grenzfrequenz f_c:

$$\alpha(f_c, T_e) = \frac{T_e}{T_e + \frac{1}{2\pi f_c}} \tag{53}$$

7.2 Exkurs: Frequenzbereich und Grenzfrequenz (f_c)

In der Signalverarbeitung unterscheidet man bei zeitdiskreten Messsignalen (wie Pixelkoordinaten aus *xfeat*) zwischen niederfrequenten Signalanteilen (langsame, bewusste Bewegungen des Objekts) und hochfrequenten Signalanteilen (schnelles Messrauschen oder Jitter).

Ein Tiefpassfilter dämpft Frequenzen, die oberhalb einer bestimmten Schwelle liegen, und lässt Frequenzen unterhalb dieser Schwelle passieren. Diese Schwelle wird als **Grenzfrequenz** f_c (Cutoff Frequency) bezeichnet (üblicherweise definiert als der $-3\,\text{dB}$-Punkt im Amplitudengang).

Im Kontext des Objekt-Trackings bestimmt f_c den fundamentalen Trade-off des Filters:

- **Niedriges f_c:** Starke Dämpfung von hochfrequentem Rauschen (eliminiert Jitter), führt jedoch zu einer starken Phasenverschiebung bzw. Signalverzögerung (Lag) bei plötzlichen Bewegungen.
- **Hohes f_c:** Geringe Phasenverschiebung (minimiert Lag), lässt jedoch hochfrequentes Sensorrauschen ungefiltert in das Regelsystem durch.

Der 1€ Filter löst diesen Konflikt, indem er f_c nicht als Konstante definiert, sondern sie dynamisch in Abhängigkeit der aktuellen Signaländerungsrate moduliert.

7.3 Dynamische Grenzfrequenz

Die Kerninnovation des Filters besteht darin, die Grenzfrequenz f_c als lineare Funktion der gefilterten Änderungsrate $|\hat{\dot{x}}_i|$ zu definieren:

$$f_c = f_{min} + \beta|\hat{\dot{x}}_i| \tag{54}$$

- f_{min} (> 0): Minimale Grenzfrequenz. Dominiert bei niedrigen Geschwindigkeiten und wird minimiert, um Jitter im Stillstand zu eliminieren.

- β (> 0): Geschwindigkeits-Koeffizient. Erhöht die Grenzfrequenz bei schnellen Bewegungen, um Lag (Latenz) zu reduzieren.

Die Änderungsrate (Geschwindigkeit) wird zur Rauschunterdrückung intern ebenfalls durch einen Tiefpassfilter mit konstanter Grenzfrequenz f_{c_d} geglättet:

$$\dot{x}_i = \frac{x_i - \hat{x}_{i-1}}{T_e} \tag{55}$$

$$\hat{\dot{x}}_i = \alpha(f_{c_d}, T_e)\dot{x}_i + (1 - \alpha(f_{c_d}, T_e))\hat{\dot{x}}_{i-1} \tag{56}$$

Fazit für IBVS: Der 1€ Filter benötigt pro Signal (z. B. u- und v-Koordinate separat) nur zwei Tuning-Parameter (f_{min} und β), bewahrt eine $O(1)$-Komplexität in Laufzeit und Speicher und ist extrem robust gegen verrauschte Keypoint-Matcher.

8 Praktisches Tuning des 1€ Filters

Ein wesentlicher Vorteil des 1€ Filters gegenüber stochastischen Schätzern (wie dem Kalman-Filter) ist sein entkoppeltes und intuitives Tuning-Verfahren. Die Parameter f_{min} (minimale Grenzfrequenz) und β (Geschwindigkeits-Koeffizient) können systematisch in zwei unabhängigen Schritten am realen System appliziert werden, wie von den Autoren [3] vorgeschlagen.

8.1 Schritt 1: Minimierung des Jitters (Statische Phase)

Ziel des ersten Schrittes ist die Eliminierung des hochfrequenten Messrauschens (Jitter) im Stillstand.

1. Der Geschwindigkeits-Koeffizient wird initial auf $\beta = 0$ gesetzt. Dadurch wird die adaptive Komponente deaktiviert und der Filter verhält sich wie ein reiner Tiefpassfilter erster Ordnung mit der festen Grenzfrequenz f_{min}.
2. Das zu trackende Objekt (bzw. die Kamera) wird in Ruhe gehalten.
3. Der Parameter f_{min} wird ausgehend von einem Startwert (z. B. 1.0 Hz) iterativ verringert, bis der sichtbare Jitter im Ausgangssignal $\hat{x}_i$ ein für das IBVS-System tolerierbares Minimum erreicht hat. Typische Werte liegen im Bereich von 0.01 Hz bis 1.0 Hz.

8.2 Schritt 2: Minimierung des Lags (Dynamische Phase)

Durch den niedrigen Wert von f_{min} weist der Filter nun eine starke Phasenverschiebung (Lag) bei Signaländerungen auf. Ziel des zweiten Schrittes ist es, diese Latenz bei Bewegung zu eliminieren.

1. Der Parameter f_{min} wird auf dem in Schritt 1 ermittelten Wert belassen.

2. Das Objekt wird nun mit der maximal zu erwartenden Systemdynamik bewegt (schnelle, abrupte Translationen im Bildraum).

3. Der Parameter β wird ausgehend von 0 iterativ in kleinen Schritten (z. B. in Zehnerpotenzen: $0.001, 0.01, 0.1$) erhöht. Dadurch steigt die dynamische Grenzfrequenz f_c bei hohen Geschwindigkeiten an.

4. β wird so lange erhöht, bis der beobachtete Lag minimiert ist und der Filter den schnellen Bewegungen ohne merkliche Verzögerung folgen kann.

Die Grenzfrequenz für die Geschwindigkeitsfilterung (f_{c_d}) wird in der Regel konstant auf 1.0 Hz belassen, da sich dieser Wert empirisch für die meisten kinematisches Tracking-Aufgaben als robust erwiesen hat.

9 Umgang mit variierenden Feature-Mengen: Transformations-basiertes Filtern

Moderne merkmalsbasierte Detektoren und Matcher (wie *xfeat*, ORB oder SuperPoint) extrahieren in jedem Frame eine unterschiedliche Anzahl an Keypoints N_k. Zudem weisen diese Punkte keine zeitliche Konsistenz (feste IDs) auf. Da rekursive Schätzer (wie der Kalman-Filter oder der 1€ Filter) einen Zustandsvektor x mit konstanter Dimension voraussetzen, ist das direkte Filtern individueller Pixelkoordinaten in solchen dynamischen Szenarien nicht trivial möglich.

Die etablierte Lösung in der visuellen Servoregelung (IBVS) ist der Übergang vom Punkt-Raum in den Transformations-Raum [4], [5].

9.1 Schätzung und Filterung der Homographie-Matrix

Anstatt N_k Punkte zu filtern, wird die globale projektive Transformation zwischen dem Referenzbild (Soll-Zustand) und dem aktuellen Kamerabild geschätzt. Unter der Annahme einer planaren Szene (wie in vielen industriellen IBVS-Anwendungen) wird diese Transformation durch eine 3×3 Homographie-Matrix H beschrieben:

$$p_{curr} \sim H \cdot p_{ref} \tag{57}$$

wobei p_{curr} und p_{ref} die homogenen Pixelkoordinaten der gematchten Punkte sind.

Der Prozess gliedert sich in drei Schritte:

1. **Robuste Schätzung (Update):** In jedem Frame k wird mittels RANSAC aus den N_k verrauschten Feature-Matches eine ungefilterte Homographie-Matrix $H_{raw,k}$ berechnet.

2. **Filterung:** Der Zustandsvektor des Filters besteht nun ausschließlich aus den Elementen der Matrix H (bzw. deren 8 Freiheitsgraden). Die Dimension des Zustandsvektors ist somit konstant (unabhängig von N_k). Die

Parameter von $H_{raw,k}$ dienen als Messung z_k für den Filter, woraus eine geglättete Matrix $H_{filtered,k}$ resultiert.

3. **Projektion (Proxy-Punkte):** Um den Fehlervektor für das IBVS-Regelgesetz zu bilden, wird die feste Menge der M ursprünglich definierten Soll-Features p^*_{ref} (die *Desired Features*) mittels der geglätteten Homographie $H_{filtered,k}$ in das aktuelle Bild projiziert.

Dieser Ansatz entkoppelt die stochastische Natur des Feature-Matchers (verschwindende Punkte, variierende Ausreißer-Raten) von der mathematischen Striktheit des Zustandsfilters und garantiert ein glattes Steuersignal konstanter Dimension für den Regler.

10 Implementierung des Standard Kalman Filters (SKF) im IBVS-Kontext

Die Anwendung eines Standard Kalman Filters (SKF) auf visuelle Servoing-Aufgaben bringt zwei fundamentale Herausforderungen mit sich: die Nichtlinearität der Projektionskinematik und die zeitlich variierende Dimension des Messvektors bei Feature-basierten Ansätzen.

10.1 Nichtlinearität der Bildraum-Kinematik

Der SKF setzt ein lineares Systemmodell ($x_k = Fx_{k-1}+Bu_k$) voraus. Die Transformation von 3D-Kamerageschwindigkeiten (Twist) $v_c = [v_x, v_y, v_z, \omega_x, \omega_y, \omega_z]^T$ in 2D-Pixelgeschwindigkeiten $\dot{p} = [\dot{u}, \dot{v}]^T$ wird durch die Interaktionsmatrix L_s beschrieben. Wie in den Grundlagen zum IBVS gezeigt, enthält diese Matrix Terme wie u^2, v^2 und uv. Selbst bei der Annahme einer konstanten und bekannten Tiefe Z verbleibt das System aufgrund der Rotationskomponenten hochgradig nichtlinear. Eine exakte physikalische Modellierung der Roboterkinematik ist im SKF daher unmöglich. Als Kompromiss und Baseline für spätere nichtlineare Filter wird für den SKF ein entkoppeltes, rein kinematisches 2D-Constant-Velocity (CV) Modell im Bildraum angenommen. Der Steuerbefehl des Roboters (u_k) wird dabei vernachlässigt.

10.2 Das Konzept der Virtuellen Proxy-Punkte

Um das Problem der variierenden Anzahl an Keypoints N_k zu umgehen (welches eine konstante Dimension des Zustandsvektors verhindert), wird ein indirekter Filteransatz gewählt.

Anstatt die N_k variablen Keypoints oder die 9 physikalisch schwer interpretierbaren Parameter der Homographie-Matrix H direkt zu filtern, nutzt der SKF vier feste, künstliche *virtuelle Proxy-Punkte* (z. B. die vier Bildecken). Der Zustandsvektor $x \in \mathbb{R}^{16}$ umfasst die 2D-Positionen und 2D-Geschwindigkeiten dieser vier Punkte:

$$x = [u_1, v_1, u_2, v_2, u_3, v_3, u_4, v_4, \dot{u}_1, \dot{v}_1, \dot{u}_2, \dot{v}_2, \dot{u}_3, \dot{v}_3, \dot{u}_4, \dot{v}_4]^T \tag{58}$$

Der Update-Zyklus verläuft wie folgt:

1. **Messungsgenerierung:** Aus den N_k Roh-Matches des Detektors (z. B. *xfeat*) wird eine ungefilterte Homographie H_{raw} berechnet. Die vier virtuellen Referenzpunkte werden mit H_{raw} projiziert. Das Ergebnis bildet den Messvektor $z_k \in \mathbb{R}^8$.

2. **Filterung:** Der SKF filtert diese vier projizierten Punkte unter Annahme des 2D-CV-Modells.

3. **Rücktransformation:** Aus den vier *gefilterten* Positionen im Zustandsvektor wird eine neue, bereinigte Homographie $H_{filtered}$ berechnet. Mit dieser werden schlussendlich die M Ziel-Features für den IBVS-Regler projiziert.

Dieser Ansatz garantiert einen konstanten, niedrigdimensionalen Zustandsraum und eine physikalisch sinnvolle Parametrisierung des Prozessrauschens Q auf den Pixelgeschwindigkeiten.

11 Der Image-Space Extended Kalman Filter (EKF)

Um das Divergenz- und Drift-Problem des SKF zu lösen, muss die Steuerungskinematik des Roboters in das Prädiktionsmodell integriert werden. Anstatt die 6-DoF 3D-Pose der Kamera als Zustand zu definieren, was eine fehleranfällige Rückprojektion der 2D-Features erfordern würde, wählt man in IBVS-Architekturen häufig den *Image-Space EKF* (Feature-based EKF).

Der Zustandsvektor $x \in \mathbb{R}^8$ beinhaltet weiterhin ausschließlich die 2D-Pixelkoordinaten der 4 virtuellen Proxy-Punkte. Die Nichtlinearität wird in das Systemmodell (Prädiktion) verlagert.

11.1 Nichtlineare Prädiktion mittels Interaktionsmatrix

Das Systemmodell $f(x, u)$ nutzt den bekannten 3D-Geschwindigkeits-Twist der Kamera v_{cam} als Eingangsgröße u_k. Die resultierende Pixelgeschwindigkeit $\dot{s}$ der Proxy-Punkte wird über die Bild-Jacobi-Matrix (Interaktionsmatrix) $L_s(x, Z)$ berechnet:

$$\dot{s} = L_s(x, Z) \cdot v_{cam} \tag{59}$$

Die nichtlineare Zustandsprädiktion erfolgt durch zeitliche Integration:

$$x_{k|k-1} = x_{k-1|k-1} + \dot{s} \cdot \Delta t \tag{60}$$

Da $f(x, u)$ nichtlinear in x ist, muss die System-Kovarianzmatrix P über die Taylor-Linearisierung 1. Ordnung aktualisiert werden:

$$P_{k|k-1} = F_k P_{k-1|k-1} F_k^T + Q \tag{61}$$

Wobei $F_k = \frac{\partial f}{\partial x}$ die System-Jacobi-Matrix ist. In der Praxis wird F_k oft numerisch mittels zentraler Differenzen approximiert, um die komplexe analytische Ableitung der Interaktionsmatrix zu umgehen.

11.2 Lineares Mess-Update

Da der Zustandsvektor x bereits direkt die projizierten 2D-Pixelkoordinaten enthält und die Messung z_k ebenfalls aus diesen 2D-Koordinaten (gewonnen über die Homographie H_{raw}) besteht, ist das Messmodell $h(x) = x$ streng linear. Folglich vereinfacht sich die Mess-Jacobi-Matrix H_k zur Einheitsmatrix $I_{8\times 8}$. Der Update-Schritt des EKF entspricht somit exakt dem des SKF, profitiert jedoch von einer drastisch verbesserten Prädiktion und erlaubt dadurch ein wesentlich aggressiveres Tuning (kleineres Q).

11.3 Wissenschaftliche Begründung des Image-Space EKF

Die Entscheidung, den Zustandsvektor im 2D-Bildraum zu belassen und die Nichtlinearität der Projektion über die Interaktionsmatrix L_s in das Systemmodell (Prädiktion) zu integrieren, ist in der Literatur als *Feature-based Tracking* oder *Image-Space Filtering* etabliert.

Dieser Ansatz bietet gegenüber der 3D-Posenschätzung signifikante Stabilitätsvorteile, da die Rückprojektion von verrauschten 2D-Pixeln in den 3D-Raum ein schlecht gestelltes Problem (ill-posed problem) darstellt, welches stark von Kalibrierfehlern beeinflusst wird [6].

Durch die direkte Nutzung des Roboter-Twists v_{cam} als Steuergröße u_k und der Interaktionsmatrix als Systemfunktion [7] wird die Steuerungskinematik des Roboters exakt abgebildet. Dies eliminiert den Filter-Lag (Verzögerung) bei plötzlichen Beschleunigungen, welcher bei naiven Constant-Velocity-Modellen unweigerlich auftritt [8]. Der EKF agiert in dieser Architektur prädiktiv, wodurch das Mahalanobis-Gating wesentlich engere Toleranzgrenzen (kleineres Messrauschen R) zulässt und die Robustheit gegen Ausreißer (False Matches) drastisch steigt.

12 Stabilität und Singularitätsvermeidung im IBVS

Bei der Ausführung von Visual-Servoing-Aufgaben im geschlossenen Regelkreis (Closed-Loop) können sowohl auf Sensor- als auch auf Aktorebene mathematische Singularitäten auftreten, die ohne Schutzmechanismen zur Divergenz des Filters oder zu extremen Gelenkgeschwindigkeiten führen.

12.1 Vermeidung von Homographie-Singularitäten (Fluchtlinie)

Die Verwendung der äußersten Bildecken als virtuelle Proxy-Punkte für den Zustandsfilter birgt das Risiko perspektivischer Singularitäten. Bei starken Ka-

merarotationen (Pitch/Yaw) relativ zur planaren Szene kann die Bildebene die Fluchtlinie (Vanishing Line) der projizierten Ebene kreuzen. Punkte, die exakt auf oder jenseits der Fluchtlinie liegen, werden durch die Homographie-Gleichung auf Unendlich abgebildet, was den Filterzustand augenblicklich destabilisiert. Die Lösung besteht darin, die virtuellen Proxy-Punkte räumlich zu konzentrieren (z. B. als verkleinertes Rechteck im Bildzentrum). Dadurch wird die Wahrscheinlichkeit, dass die Proxy-Punkte bei extremen Betrachtungswinkeln die Fluchtlinie kreuzen, drastisch minimiert.

12.2 Aktor-Singularitäten und Damped Least Squares (DLS)

Der IBVS-Regler berechnet den erforderlichen Kamera-Twist v_{cam}, welcher anschließend über die inverse Roboter-Jacobi-Matrix J^{-1} in Gelenkgeschwindigkeiten $\dot{q}$ transformiert wird. Nähert sich der Roboter einer kinematischen Singularität (z. B. Armstreckung), verliert die Matrix J ihren vollen Rang, und die Werte der Standard-Pseudoinversen J^{+} divergieren gegen Unendlich.

Um die resultierenden unkontrollierbaren Gelenkgeschwindigkeiten zu unterdrücken, wird die Standard-Pseudoinverse durch die *Damped Least Squares* (DLS) Methode (auch Levenberg-Marquardt-Verfahren) substituiert [9]. Hierbei wird ein Dämpfungsfaktor λ_{damp} eingeführt:

$$J_{dls}^{+} = J^T \left(JJ^T + \lambda_{damp}^2 I \right)^{-1} \tag{62}$$

Die DLS-Invertierung garantiert eine obere Schranke für die berechneten Geschwindigkeiten auf Kosten einer minimalen, kontrollierbaren Abweichung in der exakten Trajektorienverfolgung in der Nähe von Singularitäten. Analog wird DLS auch bei der Invertierung der Bild-Interaktionsmatrix L_s eingesetzt.

13 Literatur

- KF und EKF:
  - Das Original-Paper von Kalman, in dem der Filter erstmals vorgestellt wurde [10].
  - Das beste Tutorial für den Einstieg, Pflichtlektüre [11].
  - Das absolute Standardwerk als Buch. Kapitel 3 behandelt KF, EKF und UKF extrem detailliert [12].
- UKF:
  - Das Original-Paper zur Unscented Transform [13].
- ESKF:
  - Das Standardwerk für den ESKF, insbesondere im Kontext von IMU-gestützter Navigation [14].

  - Dies ist das wichtigste Paper für dich, wenn du den ESKF mit Quaternionen implementieren willst. Es erklärt die Kettenregel und Matrizen Schritt für Schritt [15].

- Quaternionen und 3D-Rotationen:
  - Das Standardwerk für Quaternionen, insbesondere die Kapitel über Kinematik und Fehlerzustände [16].
  - Kapitel über Kalman-Filter und 3D-Rotationen, mit vielen praktischen Beispielen [17].

- IBVS:
  - Das Standardwerk für IBVS, um den Gesamtkontext deines Projekts wissenschaftlich zu belegen [7].

- 1€ Filter:
  - Das Original-Paper, das die Idee des adaptiven Tiefpassfilters vorstellt [3].

- Image-Space EKF:
  - Dieses Paper etablierte die Nutzung von Interaktionsmatrizen für das Tracking und Filtern [6].
  - Das Standardwerk für die Herleitung der Interaktionsmatrix L_s, die im Prädiktionsschritt genutzt wird [7].
  - Analysiert die Latenzen in IBVS-Systemen und begründet die Notwendigkeit kinematischer Vorsteuerung im Filter [8].

Literatur

[1] Y. Bar-Shalom, X. R. Li und T. Kirubarajan, *Estimation with Applications to Tracking and Navigation: Theory Algorithms and Software*. New York, NY, USA: John Wiley & Sons, 2001.

[2] B. Williams, G. Klein und I. Reid, „Real-time SLAM relocalisation," in *Proceedings of the 11th IEEE International Conference on Computer Vision (ICCV)*, IEEE, 2007.

[3] G. Casiez, N. Roussel und D. Vogel, „1€ filter: a simple speed-based low-pass filter for noisy input in interactive systems," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, ACM, 2012.

[4] E. Malis, F. Chaumette und S. Boudet, „2 1/2 D visual servoing," *IEEE Transactions on Robotics and Automation*, Jg. 15, Nr. 2, S. 238–250, 1999.

[5] S. Benhimane und E. Malis, „Homography-based 2D visual tracking and servoing," *The International Journal of Robotics Research*, Jg. 26, Nr. 7, S. 661–676, 2007.

[6] A. I. Comport, E. Marchand, M. Pressigout und F. Chaumette, „Real-time markerless tracking for augmented reality: the virtual visual servoing framework," *IEEE Transactions on Visualization and Computer Graphics*, Jg. 12, Nr. 4, S. 615–628, 2006.

[7] F. Chaumette und S. Hutchinson, „Visual servo control. I. Basic approaches," *IEEE Robotics & Automation Magazine*, Jg. 13, Nr. 4, S. 82–90, 2006.

[8] P. I. Corke und M. C. Good, „Dynamic effects in visual closed-loop systems," *IEEE Transactions on Robotics and Automation*, Jg. 12, Nr. 5, S. 671–683, 1996.

[9] C. W. Wampler, „Manipulator inverse kinematic solutions based on damped least-squares solutions," *IEEE Transactions on Systems, Man, and Cybernetics*, Jg. 16, Nr. 1, S. 93–101, 1986.

[10] R. E. Kalman, „A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, Jg. 82, Nr. 1, S. 35–45, 1960.

[11] G. Welch und G. Bishop, „An Introduction to the Kalman Filter," University of North Carolina at Chapel Hill, Chapel Hill, NC, USA, Techn. Ber., 2006.

[12] S. Thrun, W. Burgard und D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.

[13] S. J. Julier und J. K. Uhlmann, „New extension of the Kalman filter to nonlinear systems," in *Signal Processing, Sensor Fusion, and Target Recognition VI*, SPIE, Bd. 3068, 1997, S. 182–193.

[14] C. Forster, L. Carlone, F. Dellaert und D. Scaramuzza, „On-Manifold Preintegration for Real-Time Visual–Inertial Odometry," *IEEE Transactions on Robotics*, Jg. 33, Nr. 1, S. 1–21, 2017.

[15] J. Solà, „Quaternion kinematics for the error-state Kalman filter," *arXiv preprint arXiv:1711.02508*, 2017.

[16] J. B. Kuipers, *Quaternions and Rotation Sequences: A Primer with Applications to Orbits, Aerospace, and Virtual Reality*. Princeton, NJ, USA: Princeton University Press, 1999.

[17] P. Corke, *Robotics, Vision and Control: Fundamental Algorithms in MATLAB* (Springer Tracts in Advanced Robotics), 2nd. Springer, 2017, Bd. 118.